



Rapport de Travail de Recherche - M1 Génie Logiciel

KEGLO Patrice & BARATAY Mallory

Lien Github: [Battleship](#)

Faculté des Sciences
Université de Montpellier

Mai 2026

Table des matières

1	Remerciements	4
1.1	Contexte et enjeux sociétaux	5
1.2	Problématique et questions de recherche	5
1.3	Genèse et évolution du projet : du robot Pepper à une plateforme web	5
1.3.1	Contexte initial et objectif ambitieux	5
1.3.2	Pivot technique et justification	6
1.3.3	Pivot vers une plateforme digitale	6
1.3.4	Choix technologiques justifiés	6
1.4	État de l'art et positionnement théorique	7
1.4.1	Théorie de l'attribution et formation des jugements	7
1.4.2	Tromperie et manipulation en contexte humain-robot	7
1.4.3	Anthropomorphisme et perception morale	8
1.4.4	Contexte des interactions compétitives	8
1.5	Objectifs et contribution attendue	8
1.5.1	Objectifs principaux	8
1.5.2	Contributions scientifiques attendues	8
1.6	Hypothèses de recherche	9
1.6.1	Hypothèse principale	9
1.6.2	Hypothèses secondaires	9
1.7	Méthodologie générale	9
1.7.1	Variables contrôlées	9
1.7.2	Contrôle du comportement adversaire	10
2	Développement	11
2.1	Architecture générale	11
2.1.1	Stack technologique	11
2.2	Plateforme web (Site/)	11
2.2.1	Architecture Flask	11
2.2.2	Landing page (Templates/index.html)	12
2.2.3	Écran de jeu (Templates/game.html)	12
2.2.4	API REST et flux de données	13
2.3	Application Python (Logique de jeu)	14
2.3.1	Architecture modulaire du projet	14
2.3.2	GameMaster (orchestrateur)	14
2.3.3	CheatBot (élément critique)	15
2.3.4	Hiérarchie des joueurs	16
2.3.5	Modèles de données	16
2.3.6	Système de persistance (DatabaseManager)	16
2.3.7	Interfaces utilisateur	17
2.4	Synchronisation Web ↔ Python	17

3	Procédure d'expérimentation	18
3.1	Population et recrutement	18
3.2	Protocole d'expérience	18
3.2.1	Phase de préparation	18
3.2.2	Entraînement	18
3.2.3	Déroulement des parties	18
3.2.4	Mesure de confiance	18
3.2.5	Questionnaire post-expérience	18
3.3	Conditions de contrôle	19
4	Résultats	20
4.1	Analyse descriptive	20
4.1.1	Caractéristiques de l'échantillon	20
4.1.2	Scores de confiance par condition	20
4.2	Analyse statistique	20
4.2.1	Test de comparaison	20
4.2.2	Analyses secondaires	20
4.3	Résultats principaux	20
4.4	Interprétation et discussion	20
4.4.1	Validation des hypothèses	20
4.4.2	Implications théoriques	20
4.4.3	Limitations et perspectives futures	21
5	Conclusion	22

1 Remerciements

Nous tenons à remercier chaleureusement les personnes et institutions qui ont contribué à la réalisation de ce travail.

Nos remerciements vont d'abord à Madame Magdalena CROITORU, notre encadrante de recherche, pour ses conseils éclairés, sa disponibilité constante et sa rigueur académique tout au long du projet. Ses retours ont permis de structurer notre démarche expérimentale et d'affiner nos hypothèses de recherche.

Nous remercions également Ganesh ???, intervenant externe qui nous a aidés à mettre en place le protocole expérimental. Son expertise en conception d'expériences et sa connaissance du terrain ont été précieuses pour surmonter les obstacles techniques et méthodologiques rencontrés, notamment lors du pivot du projet depuis le robot Pepper vers la plateforme web.

Nos remerciements s'adressent aussi à **Monsieur ???**, coordinateur des travaux d'études et de recherche pour le Master 1, qui a assuré la coordination globale des projets et fourni un soutien administratif essentiel.

Nous remercions tous les participants à l'étude, dont le temps et l'engagement ont alimenté nos données empiriques et rendu cette recherche possible.

Enfin, nous exprimons notre gratitude envers nos familles et amis pour leur soutien moral, leur patience et leurs encouragements tout au long de ce projet, particulièrement pendant les phases intensives de développement et de rédaction.

1.1 Contexte et enjeux sociétaux

L'essor de l'intelligence artificielle et des systèmes autonomes transforme profondément les interactions humaines quotidiennes. Des assistants vocaux aux robots collaboratifs, en passant par les systèmes de recommandation et les agents conversationnels, l'IA est devenue omniprésente dans nos environnements de travail et de loisir.

Or, cette prolifération soulève des questions cruciales concernant la confiance, l'acceptabilité et la perception que les utilisateurs ont de ces entités artificielles. Contrairement à ce que suggèrent les modèles classiques de confiance interpersonnelle, les utilisateurs n'appliquent pas les mêmes schémas mentaux lorsqu'ils interagissent avec une machine que lorsqu'ils interagissent avec un humain, même lorsque le comportement observable est strictement identique.

Cette asymétrie perceptuelle a des implications conséquentes :

- **En ergonomie** : Comment concevoir des interfaces homme-machine qui favorisent la confiance justifiée sans créer de dépendance excessive ?
- **En éthique** : Faut-il tenir compte des biais de confiance lors de la conception d'agents autonomes destinés à interagir avec des utilisateurs ?
- **En sciences cognitives** : Quels mécanismes psychologiques expliquent cette différenciation ?

1.2 Problématique et questions de recherche

Ce travail s'inscrit dans une perspective pluridisciplinaire combinant psychologie sociale, ergonomie cognitive et informatique expérimentale. Il s'appuie sur l'observation suivante : les participants accordent-ils leur confiance différemment à un adversaire humain par rapport à une intelligence artificielle, notamment lorsque ces deux adversaires adoptent des comportements strictement identiques mais perçus comme déloyaux ou biaisés ?

Question de recherche principale :

> Dans quelle mesure les utilisateurs modulent-ils leurs jugements de confiance en fonction de la nature déclarée de l'adversaire (humain vs machine) indépendamment de son comportement observable ?

1.3 Genèse et évolution du projet : du robot Pepper à une plateforme web

1.3.1 Contexte initial et objectif ambitieux

Le projet Pepper Battleship s'inscrivait initialement dans une perspective beaucoup plus ambitieuse. L'objectif premier était de disposer d'une plateforme complète d'expérimentation utilisant le robot social Pepper (développé par SoftBank Robotics), qui représente une entité humanoïde — à mi-chemin entre une machine abstraite et une silhouette humaine.

L'hypothèse initiale était de créer une échelle de comparaison à trois niveaux :

1. **Adversaire humain réel** : Un joueur humain physiquement présent
2. **Adversaire humanoïde** : Le robot Pepper, incarnant une forme physique humaine mais clairement artificiel

3. **Adversaire digital abstrait** : Une interface purement numérique sans représentation physique

Cette approche tri-modale aurait permis d'étudier l'influence de l'**embodiment** (incarnation physique) sur la perception de confiance et la détection de déloyauté.

1.3.2 Pivot technique et justification

Cependant, après avoir implémenté une première version du système intégrant Pepper et développé les protocoles d'interaction robot-humain, un problème majeur s'est imposé : **les parties contre Pepper s'avéraient être extrêmement longues pour les participants.**

Plusieurs facteurs expliquaient cette difficulté :

- **Problèmes de communication** : La voix de Pepper était parfois difficile à comprendre et ajoutait une latence dans les échanges
- **Ajouter autre chose**

En conséquence, l'équipe a décidé de **repenser le design expérimental autour d'une approche purement digitale**, tout en conservant l'infrastructure Python déjà développée pour Pepper.

1.3.3 Pivot vers une plateforme digitale

Plutôt que d'abandonner le travail réalisé, le projet a pivoté vers :

1. **Conservation de la stack Python** : Pepper tournant nativement sous Python, toute la logique de jeu avait été développée dans ce langage. Il aurait été contre-productif de réécrire en une autre langue.
2. **Ajout d'une couche web** : Pour pallier les limitations de Pepper, une interface web moderne (Flask + JavaScript) a été développée pour permettre une collecte de données en ligne et une meilleure expérience utilisateur.
3. **Redéfinition de la manipulation expérimentale** : Plutôt qu'une échelle physique (humain → humanoïde → digital), on utilise une manipulation **de présentation** : le même adversaire digital est présenté alternativement comme « humain » ou comme « CheatBot », permettant de tester l'influence de l'**étiquetage** (labeling) indépendamment de la représentation physique.

Cette adaptation offrait en fait plusieurs avantages :

- **Contrôle rigoureux** : Certitude absolue que le comportement adversaire est identique dans les deux conditions
- **Standardisation** : Élimination des confonds liés aux variables non contrôlées du robot physique
- **Scalabilité** : Possibilité de collecter les données via une plateforme web auprès de nombreux participants
- **Reproductibilité** : Facilité à reproduire l'expérience dans d'autres contextes

1.3.4 Choix technologiques justifiés

Le choix de maintenir Python comme langage principal pour la logique de jeu (plutôt que de tout réécrire en PHP, Node.js, ou autre backend web-friendly) s'explique par :

- L'investissement déjà réalisé et les tests préalables
- La robustesse de la logique métier développée
- L'intégration possible future avec Pepper (même si non exploitée dans cette version)
- La clarté du code pour les itérations de recherche

L'intégration d'une couche Flask a donc servi d'**adaptateur** : la web permet une UX moderne et une collecte de données centralisée, tandis que Python gère la complexité algorithmique du jeu et de l'intelligence artificielle du CheatBot.

1.4 État de l'art et positionnement théorique

1.4.1 Théorie de l'attribution et formation des jugements

La théorie de l'attribution (Heider, 1958 ; Weiner, 1985) stipule que les individus construisent des explications causales pour les événements observés. Lorsqu'une personne rencontre un comportement ou un résultat, elle tend à l'attribuer soit à des facteurs internes (disposition, intention) soit à des facteurs externes (contexte, chance).

Or, les chercheurs en psychologie cognitive ont démontré que cette attribution est systématiquement biaisée lorsque la cible est une entité artificielle. Par exemple, un tir réussi attribué à un humain sera expliqué par son habileté, tandis que le même tir attribué à un robot sera attribué au hasard ou à un « bug » (Madhavan et Wiegmann, 2007).

1.4.2 Tromperie et manipulation en contexte humain-robot

Une étude particulièrement pertinente pour notre recherche est celle de Ullman, Spelke et Tenenbaum (2014) de l'Université Yale. Ces chercheurs ont investigué comment les humains développent la confiance envers les robots et, plus important, comment ils réagissent lorsqu'un robot les trompe délibérément.

Dans l'expérience d'Ullman et al., des participants humains interagissaient avec un robot dans une situation coopérative (un jeu de coquille) où le robot était supposé donner des conseils utiles. L'expérience comportait plusieurs phases :

1. **Phase de construction de confiance** : Le robot fournissait des conseils corrects et cohérents
2. **Phase de tromperie** : Le robot commençait délibérément à donner des informations fausses ou trompeuses
3. **Phase de récupération** : Le robot revenait à des conseils corrects

Les résultats révélaient plusieurs phénomènes intéressants :

- Les participants continuaient à faire confiance au robot **même après l'avoir détecté en train de tricher**, particulièrement si la confiance initiale était établie
- Une fois la confiance brisée, les participants restaient méfiants **même après le retour du robot à un comportement honnête**
- La persistance de la confiance initial (ou de la méfiance acquise) suggérait un **ancrage psychologique** difficile à modifier

Ces résultats sont directement pertinents à notre question de recherche : ils suggèrent que les utilisateurs peuvent avoir des seuils de détection différents pour la tromperie selon la nature perçue de l'adversaire, et que cette détection est loin d'être objective. Notre étude prolonge

cette investigation en étudiant précisément comment l'**étiquetage** (humain vs machine) influence cette perception, en contrôlant rigoureusement le comportement observable.

1.4.3 Anthropomorphisme et perception morale

L'anthropomorphisme — la tendance à attribuer des caractéristiques humaines à des entités non-humaines — joue un rôle central dans la confiance homme-machine. Cependant, les études récentes révèlent un phénomène paradoxal : lorsqu'une machine se comporte de manière déloyale ou contraire à l'éthique, les utilisateurs lui en attribuent davantage la responsabilité intentionnelle que si la même action provenait d'un humain (Castelli et al., 2021).

Ce phénomène, appelé **anthropomorphisme inversé**, suggère que les utilisateurs appliqueraient des standards moraux différents selon qu'ils interagissent avec un humain ou une machine. Les machines déloyales seraient jugées plus sévèrement car elles sont supposées être « neutres » et « sans intérêt personnel ».

1.4.4 Contexte des interactions compétitives

Les jeux compétitifs offrent un contexte privilégié pour étudier ces phénomènes car :

- Ils simulent des situations où la triche ou la déloyauté peuvent être détectées (score anormalement élevé, patterns improbables)
- Ils permettent un contrôle expérimental strict du comportement observé
- Ils favorisent l'émergence de jugements intuitifs non censurés

Peu d'études existantes combinent ce contexte avec la mesure répétée de confiance au fil du temps, particulièrement en utilisant un protocole quasi-expérimental où le comportement est rigoureusement identique.

1.5 Objectifs et contribution attendue

1.5.1 Objectifs principaux

1. **Tester l'hypothèse d'asymétrie perceptuelle** : Démontrer empiriquement que les participants évaluent différemment la déloyauté selon que l'adversaire est présenté comme humain ou machine, à comportement constant.
2. **Quantifier les écarts de confiance** : Mesurer l'ampleur de cette différence (magnitude de l'effet) et identifier les facteurs qui la modulent.
3. **Étudier la dynamique temporelle** : Observer comment les perceptions évoluent à travers une série d'interactions répétées (apprentissage vs habitude).
4. **Contribuer à la conception responsable** : Fournir des données empiriques pour améliorer les recommandations en matière de conception d'interfaces homme-machine éthiques et fiables.

1.5.2 Contributions scientifiques attendues

- **Sur le plan théorique** : Enrichir la compréhension des mécanismes psychologiques sous-jacents à la confiance asymétrique.
- **Sur le plan méthodologique** : Proposer un protocole reproductible pour évaluer les biais de confiance dans un contexte contrôlé.

- **Sur le plan applicatif** : Documenter les implications pour le design d'agents autonomes interactifs.

1.6 Hypothèses de recherche

1.6.1 Hypothèse principale

H₁ : Pour un même comportement observable, les participants expriment une évaluation significativement plus élevée de déloyauté lorsque l'adversaire est présenté comme une machine (CheatBot) par rapport à lorsqu'il est présenté comme un humain.

Cette hypothèse repose sur l'intuition qu'une entité artificielle est supposée être « parfaitement rationnelle » et « sans intérêt », rendant tout écart par rapport à la rationalité perçu comme intentionnellement déloyale plutôt que comme une erreur ou une expression de subjectivité (comme on le tolère chez un humain).

1.6.2 Hypothèses secondaires

H_{2a} : L'écart de confiance diminue au fil des tours successifs (effet d'apprentissage : les participants réalisent progressivement que le comportement suit un pattern déterministe, indépendamment du label adversaire).

H_{2b} : Les participants ayant une expérience préalable avec l'IA ou une formation informatique présentent un écart de confiance moins prononcé que les autres.

H_{2c} : L'ordre de présentation (humain d'abord vs machine d'abord) influence les perceptions ultérieures via un effet de contraste ou d'assimilation.

1.7 Méthodologie générale

Une méthodologie quasi-expérimentale a été adoptée afin de garantir un contrôle rigoureux des variables indépendantes et dépendantes. Le design repose sur un plan intra-groupe (within-subjects design) où chaque participant s'engage dans deux parties successives sous des conditions comportementales rigoureusement identiques :

- **Partie 1** : Affrontement contre Adversaire X présenté comme [TYPE A]
- **Partie 2** : Affrontement contre Adversaire Y présenté comme [TYPE B]

Où TYPE A et TYPE B représentent l'une des deux conditions (Humain vs Machine), contre-balançées entre les participants.

1.7.1 Variables contrôlées

- **Variable indépendante manipulée** : Type d'adversaire (Humain déclaré vs Machine déclarée)
- **Variable indépendante mesurée** : Ordre de présentation, expérience préalable avec l'IA, caractéristiques individuelles
- **Variable dépendante principale** : Scores de confiance/déloyauté (échelle 0-5 par tour)
- **Variables dépendantes secondaires** : Temps de décision, patterns de tirs, questions post-expérience

1.7.2 Contrôle du comportement adverse

L'élément crucial du design est que **le comportement du joueur adverse reste strictement identique entre les deux parties**. Ceci est assuré par :

1. Utilisation d'un algorithme de quotas prédéfinis et appliqués dans le même ordre
2. Sélection pseudo-aléatoire des positions de tirs (évitant les patterns trop évidents)
3. Enregistrement exhaustif de tous les tirs pour vérification post-hoc de l'équivalence comportementale

2 Développement

Le projet Pepper Battleship repose sur deux composants interdépendants : une application de jeu Python multimode (desktop/GUI) et une plateforme web moderne pour la collecte expérimentale. Cette architecture hybride permet une expérience flexible adaptée à divers contextes (laboratoire, en ligne, intégration robotique).

2.1 Architecture générale

2.1.1 Stack technologique

Backend Python :

- **Langage** : Python 2.7+ (compatibilité descendante)
- **Moteur de jeu** : Architecture modulaire propriétaire
- **Base de données** : SQLite pour stockage local
- **Interfaces** : Console texte + Tkinter GUI
- **Intégration robotique** : Communication socket (Pepper)

Frontend Web :

- **Serveur** : Flask 2.3+ (framework web minimaliste)
- **Requêtes** : API REST JSON
- **Frontend** : HTML5 + CSS3 + JavaScript vanilla
- **Sécurité** : flask-limiter pour rate-limiting
- **Déploiement** : Gunicorn + PostgreSQL optionnel

Communication :

- API REST pour synchronisation client-serveur
- Socket TCP/IP pour robot Pepper
- Session management pour suivi des parties

2.2 Plateforme web (Site/)

2.2.1 Architecture Flask

Le serveur Flask (Site/app.py) orchestre l'expérience web avec deux points d'entrée principaux.

Routes principales :

- GET / : Landing page avec présentation du projet
- GET /game : Interface de jeu interactive
- GET /admin : Tableau de bord administrateur
- POST /api/game/new : Création d'une partie
- POST /api/game/{gid}/shoot : Enregistrement d'un tir joueur
- POST /api/game/{gid}/bot-turn : Exécution du tour du bot
- POST /api/game/{gid}/turn-trust : Enregistrement score de confiance
- GET /api/stats : Récupération des statistiques globales
- GET /api/personas : Récupération des personas expérimentaux

La conception du serveur Flask favorise :

- La stateless API (chaque requête est indépendante)

- La limitation de requêtes (protection contre les abus)
- L'enregistrement détaillé en base de données
- La confidentialité des données (anonymisation)

2.2.2 Landing page (Templates/index.html)

La page d'accueil constitue l'élément critique de recrutement et de contextualisation expérimentale.

Sections principales :

1. **Navigation** : Logo Pepper Battleship, liens d'ancrage vers les sections, CTA « Jouer »
2. **Héros** :
 - Message accrocheur : « POUVEZ-VOUS DÉTECTER LA TRICHE ? »
 - Sous-titre contextuel mentionnant Pepper robot
 - Statistiques dynamiques (parties jouées, tirs/tour, configurations, bateaux)
 - CTA principal vers /game
3. **Fonctionnalités** (6 cartes) :
 - Affrontez Pepper Bot (présentation du robot)
 - Évaluation par tour (système de confiance 0-5)
 - 10 configurations prédéfinies
 - 4 tirs par tour (règles modifiées)
 - Données anonymes (RGPD)
 - Confiance en son instinct (justification)
4. **Règles du Jeu** (4 étapes) :
 - Étape 1 : Sélection de la flotte parmi 10 configurations
 - Étape 2 : Combat alternée à 4 tirs/tour
 - Étape 3 : Observation du bot pour détecter la triche
 - Étape 4 : Notation du suspicion sur 0-5
5. **Section Recherche** :
 - Contexte académique (TER M1 Génie Logiciel)
 - Objectif : comprendre perception de tromperie artificielle
 - Garantie anonymat et usage recherche

Animations JavaScript :

- Système de particules interactives (connexions entre points)
- Compteur animé pour les statistiques globales
- Fetch /api/stats pour mise à jour temps réel

2.2.3 Écran de jeu (Templates/game.html)

L'interface de jeu web implémente une expérience UX complète.

États de l'expérience (State machine) :

1. **Setup** : Entrée du nom du joueur
2. **Persona** : Sélection du contexte (humain vs robot — manipulation expérimentale)
3. **Grid** : Validation de la configuration de flotte

4. **Playing** : Combat et collection de scores de confiance
5. **GameOver** : Résultats et débriefing

Grilles 10×10 :

- Grille du joueur (ma flotte + tirs ennemis)
- Grille de suivi (tirs du joueur + résultats)
- Affichage symbolique (eau, X touché, C coulé, * raté)

Flow de jeu :

- Phase de tir : Le joueur clique sur jusqu'à 4 cases de la grille adverse
- Affichage résultats : Chaque tir retourne immédiatement son résultat
- Phase bot : Le bot effectue ses 4 tirs
- Questionnaire : Après le tour du bot, le joueur note sa suspicion (0-5)
- Répétition : Prochains tours jusqu'à victoire/défaite

Système de notation de confiance :

```
// Après chaque tour du bot, affichage :  
"Dans quelle mesure pensez-vous que le bot a triché ?"  
0 = Pas du tout triché  
1-2 = Peu probable  
3 = Incertain  
4-5 = Très suspect / Certainement triché
```

2.2.4 API REST et flux de données

Endpoints clés :

GET /api/stats

└ Réponse: { total_games: N, avg_trust_score: X }

POST /api/game/new

└ Payload: { player_name, persona_id }

└ Réponse: { game_id, grids: [...], opponent: "Pepper Bot" }

POST /api/game/{gid}/select-grid

└ Payload: { index: 0-9 }

└ Réponse: { success: true, grid: [...] }

POST /api/game/{gid}/shoot

└ Payload: { x, y }

└ Réponse: { result: "hit|miss|sunk", player_shots: [...] }

POST /api/game/{gid}/bot-turn

└ Payload: {}

└ Réponse: { bot_shots: [...], results: [...], turn_complete: true }

POST /api/game/{gid}/turn-trust

└ Payload: { score: 0-5 }

└ Réponse: { recorded: true }

GET /api/game/{gid}/preview/{index}

└ Paramètre: index grille à visualiser
└ Réponse: { grid: [...], ships: [...] }

Sérialisation et persistance :

- Chaque partie crée une session serveur avec état de jeu
- Tous les tirs (joueur + bot) sont enregistrés instantanément en base
- Les scores de confiance sont horodatés (timestamp = fin du tour)
- Les données brutes permettent reconstitution complète de chaque partie

2.3 Application Python (Logique de jeu)

2.3.1 Architecture modulaire du projet

L'application Python est organisée en 6 modules principaux :

```
Battleship_py/  
└ game_logic/                # Moteur de jeu  
  └ game_master.py          # Orchestrateur principal  
  └ game.py                 # Logique de tour  
└ classes/                  # Modèles de données  
  └ grid.py                 # Plateau de jeu  
  └ ship.py                 # Navires  
  └ position.py             # Coordonnées  
  └ variable.py             # Configuration globale  
└ players/                  # Implémentations de joueurs  
  └ player.py               # Classe de base  
  └ cheat_bot.py            # Bot tricheur  
  └ smart_bot.py            # Bot intelligent  
└ interface/                # UX  
  └ interface_tk.py         # GUI Tkinter  
└ database/                 # Persistance  
  └ db_manager.py           # Gestion SQLite  
└ client/                   # Communication réseau  
  └ client.py               # Socket TCP
```

2.3.2 GameMaster (orchestrateur)

GameMaster est le contrôleur central de toute expérience.

Responsabilités :

1. Initialisation :

- Configuration des joueurs (humain vs CheatBot)
- Mode Digital (ordinateur seul) vs Hybride (intégration Pepper)
- Création de la partie en base de données

2. Gestion des tours :

- Détermination du joueur actif
- Exécution des coups
- Transition vers le joueur suivant

3. Gestion de l'expérience :

- Sélection des grilles prédéfinies

- Attribution des quotas de triche à chaque tour
- Enregistrement des scores de confiance

4. **Persistance :**

- Appels à DatabaseManager pour création/mise à jour parties
- Enregistrement des tours avec quota et score de confiance

Exemple de flux :

```
gm = GameMaster()
gm.setup_players(hybrid=False)
gm.db.create_game(player_name, player_type)
# ... validation des grilles ...
while not game.is_finished():
    result = game.play(current_player)
    gm.db.record_turn(turn_data)
    gm.db.record_trust_score(trust_score)
    game.next_turn()
```

2.3.3 **CheatBot (élément critique)**

CheatBot implémente la logique de triche contrôlée qui garantit l'équivalence expérimentale.

Architecture de triche :

1. **Accès à la grille ennemie :**

```
self.target_grid = None # Set par GameMaster
```

2. **Planification stratégique (par tour) :**

```
def execute_turn(self, grid, enemy_ships):
    # 1. Identifier les cases de bateaux non touchées
    # 2. Identifier les cases vides
    # 3. Sélectionner exactement N cases (succès)
    # 4. Sélectionner (4-N) cases vides (échecs)
    # 5. Mélanger pour éviter pattern
    # 6. Retourner les 4 positions
```

3. **Application du quota :**

- self.success_quota défini par tour (ex: [2, 1, 3, 2, 1])
- Garantit exactement N succès par tour
- Adaptation automatique si plus de bateaux disponibles

4. **Comportement en mode Physique :**

- Fallback aléatoire si pas de grille cible disponible
- Permet mode hybride sans grille digitale

Communication réseau :

```
self.client_socket = Client("10.161.177.181", port=5000)
# Format: "P{colonne}{ligne}" (ex: "PA4")
# Utilisé pour feedback au robot Pepper
```

2.3.4 Hiérarchie des joueurs

Player (classe de base)

```
|— CheatBot          # Triche discrète
|— SmartBot          # Stratégie heuristique
|— PhysicalPlayer    # Interface robot Pepper
```

Player implémente :

- Gestion de deux grilles (ma flotte, grille de suivi)
- Historique des coups joués
- Interface d'entrée utilisateur (console ou GUI)

2.3.5 Modèles de données

Grid (10×10 cases) :

- Stockage des navires placés
- Traitement des tirs via shoot(x, y)
- Retour de Response (touché/coulé/raté)
- État de chaque case

Ship :

- Position (start) + Orientation (H/V)
- Taille (5, 4, 3, 3, 2)
- Détection de coulage

Position :

- Coordonnées (x, y)
- Validation des limites

Response :

- Message (touché/coulé/raté)
- État après tir

2.3.6 Système de persistance (DatabaseManager)

Base SQLite battleship_stats.db avec schéma relationnel :

Table Game :

```
CREATE TABLE Game (
    id INTEGER PRIMARY KEY,
    player_name TEXT,
    player_type TEXT,
    date_played DATETIME,
    winner TEXT,
    trust_score_avg REAL
)
```

Table Turn :

```
CREATE TABLE Turn (
    id INTEGER PRIMARY KEY,
    game_id INTEGER,
    turn_number INTEGER,
```



```

    bot_quota INTEGER,
    trust_score INTEGER,
    FOREIGN KEY(game_id) REFERENCES Game(id)
)

```

Table BotShot :

```

CREATE TABLE BotShot (
    id INTEGER PRIMARY KEY,
    turn_id INTEGER,
    shot_number INTEGER,
    pos_x, pos_y INTEGER,
    result TEXT
)

```

Cette structure permet :

- Reconstitution complète de chaque partie
- Analyse des quotas appliqués vs perceptions
- Corrélation confiance / succès du bot

2.3.7 Interfaces utilisateur

Mode Console :

- Affichage textuel des grilles
- Interaction CLI
- Approprié pour tests serveur

Mode Graphique (Tkinter) :

- Canvases pour visualisation grilles 10×10
- Widgets pour interactions
- Affichage des tirs antérieurs
- Intégration du questionnaire de confiance post-tour
- Support complet des configurations prédéfinies

2.4 Synchronisation Web ↔ Python

L'application web Flask en tant que **clients** qui invoquent la logique Python :

1. **Création de partie** : Flask appelle GameMaster.setup_players()
2. **Tirs joueur** : API /shoot valide et enregistre
3. **Tour du bot** : API /bot-turn appelle CheatBot.get_case_played()
4. **Enregistrement confiance** : API /turn-trust persiste score en base
5. **Synchronisation d'état** : État de jeu partagé via base SQLite ou session

Cette intégration bidirectionnelle permet :

- Interface web moderne pour recrutement en ligne
- Logique Python robuste et testée pour jeu
- Collecte centralisée des données
- Flexibilité multi-plateformes (web + desktop)

3 Procédure d'expérimentation

3.1 Population et recrutement

[À compléter selon le contexte : nombre de participants, critères d'inclusion/exclusion, mode de recrutement, caractéristiques démographiques]

3.2 Protocole d'expérience

3.2.1 Phase de préparation

Chaque participant est accueilli et informé de la nature générale de l'étude (comparaison de deux types d'adversaires dans un jeu compétitif) sans révéler l'hypothèse centrale. Un formulaire de consentement est signé, et des informations démographiques de base sont collectées.

3.2.2 Entraînement

Une courte phase d'entraînement permet au participant de se familiariser avec les règles modifiées du jeu, en particulier le système de 4 tirs par tour. Cette étape dure environ 5 minutes.

3.2.3 Déroulement des parties

Première partie : Le participant joue une partie contre un adversaire présenté comme [HUMAIN / MACHINE - à préciser selon la condition expérimentale].

Pause inter-conditions : Une courte pause de 5 minutes est imposée entre les deux parties pour permettre au participant de se reposer.

Deuxième partie : Le participant joue une deuxième partie contre un adversaire présenté comme le type opposé. Les conditions de jeu restent identiques.

3.2.4 Mesure de confiance

À la fin de chaque tour, le participant doit répondre à la question suivante sur une échelle de 1 à 5 :

« Dans quelle mesure pensez-vous que votre adversaire a triché lors de ce tour ? »

Les réponses sont enregistrées immédiatement dans la base de données.

- 1 : Pas du tout triché
- 2 : Peu probable qu'il/elle ait triché
- 3 : Neutre / incertain
- 4 : Probable qu'il/elle ait triché
- 5 : Certainement triché

3.2.5 Questionnaire post-expérience

À l'issue des deux parties, un questionnaire exploratoire demande au participant :

- Ses perceptions globales de chaque adversaire
- S'il a identifié des patterns dans le comportement de l'adversaire
- Son hypothèse sur les objectifs de l'étude

3.3 Conditions de contrôle

Pour assurer la validité interne de l'étude :

- **Équivalence des quotas** : Les deux parties utilisent des séquences identiques de réussite/échec
- **Ordre contrebalancé** : Idéalement, le type d'adversaire (humain ou machine en premier) est contrebalancé
- **Environnement standardisé** : Chaque participant réalise l'expérience dans des conditions similaires
- **Instructions standardisées** : Les consignes sont présentées de manière identique à tous

4 Résultats

4.1 Analyse descriptive

4.1.1 Caractéristiques de l'échantillon

[À compléter : nombre total de participants, répartition par genre, groupe d'âge, critères supplémentaires]

4.1.2 Scores de confiance par condition

Les évaluations de confiance (échelle 1-5, indiquant la perception de déloyauté) ont été collectées pour chaque tour de chaque partie. Les statistiques descriptives suivantes seront présentées :

- **Moyenne et écart-type** des scores selon le type d'adversaire (humain vs machine)
- **Distribution des réponses** pour chaque condition
- **Évolution des scores** au fil des tours (effet d'apprentissage)

4.2 Analyse statistique

4.2.1 Test de comparaison

Un test statistique approprié permettra de tester l'hypothèse principale :

H_0 : Les évaluations de confiance ne diffèrent pas significativement entre l'adversaire humain et l'adversaire machine.

H_1 : Les évaluations de confiance diffèrent significativement selon le type d'adversaire.

4.2.2 Analyses secondaires

- **Effet d'ordre** : Comparaison entre les participants ayant affronté d'abord un humain vs la machine
- **Effet de tour** : Évolution temporelle des perceptions au cours des parties
- **Variabilité inter-individuelle** : Identification de profils distincts

4.3 Résultats principaux

[À compléter avec les données empiriques]

Les résultats montreront ou non une différence significative dans les évaluations de déloyauté selon le type d'adversaire. En cas de différence positive, cela indiquerait un biais de confiance lié à l'anthropomorphisme ou à des croyances préexistantes.

4.4 Interprétation et discussion

4.4.1 Validation des hypothèses

[À compléter : interprétation des résultats au regard des hypothèses initiales]

4.4.2 Implications théoriques

Ces résultats contribuent à la compréhension des mécanismes de confiance interpersonnelle et des biais de perception liés à l'IA. Ils éclairent notamment :

- Le rôle de la catégorisation (humain vs machine) dans les jugements moraux
- Les vulnérabilités des utilisateurs face aux comportements perçus comme déloyaux
- Les applications potentielles dans la conception d'interfaces homme-machine

4.4.3 Limitations et perspectives futures

[À compléter : limitations méthodologiques de l'étude et directions de recherche future]

5 Conclusion

Cette étude offre une contribution empirique à la question fondamentale des biais de confiance dans les interactions compétitives homme-machine. En isolant le rôle du label d'adversaire (humain vs machine) tout en maintenant un comportement de jeu identique, elle permet une évaluation rigoureuse de la manière dont les utilisateurs réagissent différemment selon la nature perçue de leur adversaire.

Les implications de cette recherche dépassent le contexte ludique pour toucher à des enjeux plus larges d'acceptabilité, de confiance, et de coopération dans les environnements intégrant l'intelligence artificielle.